Start spark Session –

```
From pyspark.sql import sparksession

# Initialize spark

Spark = sparksession.builder\
.appname("pyspark applications")\
Getorcreate ()
```

```
# read csv:
Employee = spark.read.csv('employee.csv',inferschema = true, header = true)
```

```
# Narrow transformation: Select a few columns

selected_data = result_with_regions.select('employee_name', 'department_name',
'region_name')



# Narrow transformation: Filter

filtered_employees = employees.filter(employees.age > 30)



# Wide transformation: Join

result = filtered_employees.join(departments, filtered_employees.dept_id ==
departments.dept_id)



# Action: Collect

result_list = result_with_regions.collect()


# Action: Save as CSV

selected_data.write.csv('result.csv')

# Stop Spark

spark.stop()
```

Parallelize the Data:

Use the `parallelize` method of `SparkContext` to create an RDD from your list of data:

```
data = [1, 2, 3, 4, 5]

rdd = sc.parallelize(data)
```

To verify that the RDD has been created correctly and to print its elements, you can use the `collect` method, which retrieves the entire dataset from the RDD to the driver program.

```
# Collect and print the RDD elements

print(rdd.collect())
```

```
# to set block size in hdfs:

dfs.blocksize
```

```
data1 = [
    (1, "Alice"),
    (2, "Bob"),
    (3, "Charlie")
]

data2 = [
    (1, "New York"),
    (2, "Los Angeles"),
    (4, "Chicago")
]

columns1 = ["id", "name"]
columns2 = ["id", "city"]

df1 = spark.createDataFrame(data1, columns1)
df2 = spark.createDataFrame(data2, columns2)

df1.show()
df2.show()
```

Inner Join:

```
inner_join_df = df1.join(df2, on="id", how="inner")
inner_join_df.show()
```

left join:

```
left_join_df = df1.join(df2, on="id", how="left")
left_join_df.show()
```

if column name is different -

```
inner_join_df = df1.join(df2, df1.id1 == df2.id2, "inner")

inner_join_df.show()
```

```
From pyspark.sql import sparksession
From pyspark.sql.type import *

# initialize spark:
Spark = sparksession.builder\
Appname('spark app')\
Getorcreate()
```

- Read csv with own schema not with inferschema

```
Airplane_data_schema = structtype([
Structfield('name','stringtype(),true),
Structfield('id',integertype(),true)])
# true means have null accept false not accept null or not nullbale
```

- To read file

2 method with own schema or inferschema

1. Own schema-

```
Airplane_data = spark.read.csv
```

Or like

```
Airplane_data = spark.read.format('csv').(file path,header= true,schema = Airplane_data_schema)
```

2. Inferschema =

   ```
   Airplane_data = spark.read.csv(fillepath,inferschema = true,header=true)
   ```

```
# to add new column (withcolumn)

Airplane_data= airplane_data.withcolumn('rank', lit(1) .cast(IntegerType()))


# drop column

Airplane_data = airplane_data.drop(airplane_data['rank'])


# to show it

Airplane_data.show()
```

Creating a DataFrame in Spark can be done in several ways, depending on the source of the data and the desired structure. Here are some common methods to create a DataFrame in Spark:

1. From a Collection (e.g., List of Tuples)

You can create a DataFrame from a collection like a list of tuples. This is useful for small datasets or testing.

```
python
Copy code
from pyspark.sql import SparkSession

# Initialize Spark session
spark = SparkSession.builder.appName("example").getOrCreate()

# Sample data
data = [("Alice", 34), ("Bob", 45), ("Cathy", 29)]
columns = ["Name", "Age"]

# Create DataFrame from a list of tuples
df = spark.createDataFrame(data, columns)

df.show()
```

2. From a Pandas DataFrame

If you have a Pandas DataFrame, you can convert it to a Spark DataFrame.

```
python
Copy code
import pandas as pd
from pyspark.sql import SparkSession

# Initialize Spark session
spark = SparkSession.builder.appName("example").getOrCreate()

# Create a Pandas DataFrame
pandas_df = pd.DataFrame({
    "Name": ["Alice", "Bob", "Cathy"],
    "Age": [34, 45, 29]
})

# Convert Pandas DataFrame to Spark DataFrame
df = spark.createDataFrame(pandas_df)

df.show()
```

3. From an RDD

If you have an RDD (Resilient Distributed Dataset), you can convert it to a DataFrame by defining the schema.

```
python
Copy code
from pyspark.sql import SparkSession
```

```
from pyspark.sql.types import StructType, StructField, StringType,
IntegerType

# Initialize Spark session
spark = SparkSession.builder.appName("example").getOrCreate()

# Sample data in RDD
rdd = spark.sparkContext.parallelize([("Alice", 34), ("Bob", 45), ("Cathy",
29)])

# Define schema
schema = StructType([
    StructField("Name", StringType(), True),
    StructField("Age", IntegerType(), True)
])

# Convert RDD to DataFrame
df = spark.createDataFrame(rdd, schema)

df.show()
```

4. From a CSV File

You can read data from a CSV file into a DataFrame.

```
python
Copy code
from pyspark.sql import SparkSession

# Initialize Spark session
spark = SparkSession.builder.appName("example").getOrCreate()

# File path to your CSV file
filepath = "path/to/your/data.csv"

# Read CSV file into DataFrame
df = spark.read.csv(filepath, inferSchema=True, header=True)

df.show()
```

5. From a JSON File

You can read data from a JSON file into a DataFrame.

```
python
Copy code
from pyspark.sql import SparkSession

# Initialize Spark session
spark = SparkSession.builder.appName("example").getOrCreate()

# File path to your JSON file
filepath = "path/to/your/data.json"

# Read JSON file into DataFrame
df = spark.read.json(filepath)

df.show()
```

6. From a Parquet File

You can read data from a Parquet file into a DataFrame.

```
python
Copy code
from pyspark.sql import SparkSession

# Initialize Spark session
spark = SparkSession.builder.appName("example").getOrCreate()

# File path to your Parquet file
filepath = "path/to/your/data.parquet"

# Read Parquet file into DataFrame
df = spark.read.parquet(filepath)

df.show()
```

7. Programmatically Specifying the Schema

You can create a DataFrame by specifying the schema programmatically.

```
python
Copy code
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType,
IntegerType

# Initialize Spark session
spark = SparkSession.builder.appName("example").getOrCreate()

# Define schema
schema = StructType([
    StructField("Name", StringType(), True),
    StructField("Age", IntegerType(), True)
])

# Sample data
data = [("Alice", 34), ("Bob", 45), ("Cathy", 29)]

# Create DataFrame with specified schema
df = spark.createDataFrame(data, schema)

df.show()
```

8. Using SQL Queries

If you have a Hive or an existing table, you can create a DataFrame by running a SQL query.

```
python
Copy code
from pyspark.sql import SparkSession

# Initialize Spark session
spark =
SparkSession.builder.appName("example").enableHiveSupport().getOrCreate()
```

```
# Create DataFrame using SQL query
df = spark.sql("SELECT * FROM your_table")

df.show()
```

These methods cover a wide range of scenarios for creating DataFrames in PySpark, allowing you to choose the one that best fits your data source and workflow.

Creating an RDD (Resilient Distributed Dataset) in Apache Spark can be done in various ways, depending on the source of the data and the requirements of your application. Here are some common methods to create an RDD:

1. Parallelizing a Collection

You can create an RDD from a collection (e.g., a list) by parallelizing it.

```
python
Copy code
from pyspark.sql import SparkSession

# Initialize Spark session
spark = SparkSession.builder.appName("example").getOrCreate()

# Sample data
data = [1, 2, 3, 4, 5]

# Create RDD by parallelizing the data
rdd = spark.sparkContext.parallelize(data)

rdd.collect()
```

2. Reading from External Storage (e.g., Files)

You can create an RDD by reading data from external storage like HDFS, S3, or local file system.

From Text Files

```
python
Copy code
# Create RDD by reading text files
rdd = spark.sparkContext.textFile("hdfs://path/to/your/file.txt")
```

From Sequence Files

```
python
Copy code
# Create RDD by reading sequence files
rdd = spark.sparkContext.sequenceFile("hdfs://path/to/your/file.seq")
```

From CSV Files

```
python
Copy code
# Create RDD by reading CSV files
rdd = spark.sparkContext.textFile("hdfs://path/to/your/file.csv")
```

From Other File Formats (e.g., JSON, Parquet)

Similarly, you can create RDDs from other file formats supported by Spark.

3. Transforming Existing RDDs

You can create new RDDs by transforming existing RDDs using transformations like `map`, `flatMap`, `filter`, etc.

```
python
Copy code
```

```
# Create RDD by transforming an existing RDD
rdd = existing_rdd.map(lambda x: x * 2)
```

4. From Pair RDDs

You can create Pair RDDs (RDDs with key-value pairs) using methods like `parallelizePairs`.

```
python
Copy code
# Sample data for Pair RDD
data = [("Alice", 34), ("Bob", 45), ("Cathy", 29)]

# Create Pair RDD
pair_rdd = spark.sparkContext.parallelizePairs(data)

pair_rdd.collect()
```

5. From External Databases

You can create RDDs by querying data from external databases using connectors like JDBC or ODBC.

```
python
Copy code
# Example using JDBC
jdbc_url = "jdbc:postgresql://your_database_url"
properties = {
    "user": "your_username",
    "password": "your_password",
    "driver": "org.postgresql.Driver"
}
query = "SELECT * FROM your_table"
jdbc_rdd = spark.sparkContext.jdbc(jdbc_url, query, properties)
```

6. From RDDs in Other Languages

If you are using PySpark along with other languages like Scala or Java, you can create RDDs from RDDs in those languages.

7. From In-Memory Data Structures (e.g., List, Set)

You can create RDDs from in-memory data structures like lists or sets.

```
python
Copy code
# Create RDD from a list
data = [1, 2, 3, 4, 5]
rdd = spark.sparkContext.parallelize(data)
```

Summary

Creating RDDs in Spark involves various methods, depending on the source of the data and the requirements of your application. By choosing the appropriate method, you can efficiently create RDDs to perform distributed data processing tasks in Spark.

Filter data:

```
# to check null present or not

Df.filter(col[‘name’].isnotnull())

# filter

Df.filter(df.rank > 5)


# similar to filter there is where method for data filter:

df.where(col("name").isNotNull)

df.where(col("name").isNull)



# Get distinct values
distinct_df = df.distinct()

# specific column

Distinct_df = df.select(‘name’).distinct()



# Filter names with length > 3
filtered_df = df.filter(length(col("name")) > 3)


# Convert names to uppercase
upper_df = df.withColumn("name", upper(col("name")))


# Cast 'id' column from String to Integer
casted_df = df.withColumn("id", col("id").cast("int"))


# Add a new column 'name_length' based on the length of 'name'
conditional_df = df.withColumn("name_length", when(length(col("name")) > 3,
"long").otherwise("short"))
```

```
# Sort by 'name' in descending order
sorted_df = df.orderBy(col("name").desc())

# Group by 'name' and count the occurrences
grouped_df = group_df.groupBy("name").count()



from pyspark.sql.functions import sum
# Aggregate by summing the 'value' column grouped by 'name'
aggregate_df = group_df.groupBy("name").agg(sum("value").alias("total_value"))
```

Display DataFrame Contents in Table

```
# 20 charactes from column value
df.show()

#Display full column contents
df.show(truncate=False)

# Display 2 rows and full column contents
df.show(2,truncate=False)

# Display 2 rows & column values 25 characters
df.show(2,truncate=25)

# Display DataFrame rows & columns vertically
df.show(n=3,truncate=25,vertical=True)
```

PySpark StructType & StructField

```
# Imports
import pyspark
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType,StructField, StringType,
IntegerType

spark = SparkSession.builder.master("local[1]") \
                    .appName('SparkByExamples.com') \
                    .getOrCreate()

data = [("James","","Smith","36636","M",3000),
     ("Michael","Rose","","40288","M",4000),
     ("Robert","","Williams","42114","M",4000),
     ("Maria","Anne","Jones","39192","F",4000),
     ("Jen","Mary","Brown","","F",-1)
   ]

schema = StructType([ \
     StructField("firstname",StringType(),True), \
     StructField("middlename",StringType(),True), \
     StructField("lastname",StringType(),True), \
     StructField("id", StringType(), True), \
     StructField("gender", StringType(), True), \
     StructField("salary", IntegerType(), True) \
   ])

df = spark.createDataFrame(data=data,schema=schema)
df.printSchema()
df.show(truncate=False)
```

PySpark Column Class

`pyspark.sql.Column` class provides several functions to work with DataFrame to manipulate the Column values, evaluate the boolean expression to filter rows, retrieve a value or part of a value from a DataFrame column, and to work with list, map & struct columns.

```
#Using SQL col() function
from pyspark.sql.functions import col
df.select(col("gender")).show()
#Accessing column name with dot (with backticks)
df.select(col("`name.fname`")).show()
```

```
#alias
from pyspark.sql.functions import expr
df.select(df.fname.alias("first_name"), \
          df.lname.alias("last_name")
    ).show()

#Another example
df.select(expr(" fname ||','|| lname").alias("fullName") \
    ).show()
```

4.2 asc() & desc() – Sort the DataFrame columns by Ascending or Descending order.

```
#asc, desc to sort ascending and descending order repsectively.
df.sort(df.fname.asc()).show()
df.sort(df.fname.desc()).show()
```

4.4 between() – Returns a Boolean expression when a column values in between lower and upper bound.

```
#between
df.filter(df.id.between(100,300)).show()
```

4.3 cast() & astype() – Used to convert the data Type.

```
#cast
df.select(df.fname,df.id.cast("int")).printSchema()
```

4.5 contains() –

Check if a PySpark DataFrame column value contains a string value specified in this function.

```
#contains
df.filter(df.fname.contains("Cruise")).show()
```

4.6 startswith() & endswith() –

Checks if the value of the DataFrame Column startsWith() and endsWith() a String. startsWith() filters rows where a specified substring exists at the beginning while `endsWith()` filter rows where the specified substring presents at the end.

```
#startswith, endswith()
df.filter(df.fname.startswith("T")).show()
df.filter(df.fname.endswith("Cruise")).show()
```

4.9 like() & rlike() – Similar to SQL LIKE expression

```
#like , rlike
df.select(df.fname,df.lname,df.id)  \
   .filter(df.fname.like("%om"))
```

4.10 substr() – Returns a Column after getting sub string from the Column

```
df.select(df.fname.substr(1,2).alias("substr")).show()
```

4.11 when() & otherwise() – It is similar to SQL Case When, executes sequence of expressions until it matches the condition and returns a value when match.

```
#when & otherwise
from pyspark.sql.functions import when
df.select(df.fname,df.lname,when(df.gender=="M","Male") \
                .when(df.gender=="F","Female") \
                .when(df.gender==None ,"") \
                .otherwise(df.gender).alias("new_gender") \
     ).show()
```

PySpark Select Columns From DataFrame

```
# Select columns by different ways
df.select("firstname","lastname").show()
df.select(df.firstname,df.lastname).show()
df.select(df["firstname"],df["lastname"]).show()
```

```
# Select All columns from List
df.select(*columns).show()
```

```
#Selects first 3 columns and top 3 rows
df.select(df.columns[:3]).show(3)
#Selects columns 2 to 4  and top 3 rows
df.select(df.columns[2:4]).show(3)
```

PySpark Collect() – Retrieve data from DataFrame

```
dataCollect = deptDF.collect()
print(dataCollect)
```

PySpark RDD/DataFrame `collect()` is an action operation that is used to retrieve all the elements of the dataset (from all nodes) to the driver node. We should use the collect() on smaller dataset usually after filter(), group() e.t.c. Retrieving larger datasets results in `OutOfMemory` error.

When to avoid Collect()

Usually, collect() is used to retrieve the action output when you have very small result set and calling `collect()` on an RDD/DataFrame with a bigger result set causes out of memory as it returns the entire dataset (from all workers) to the driver hence we should avoid calling collect() on a larger dataset.

collect () vs select ()

`select()` is a transformation that returns a new DataFrame and holds the columns that are selected whereas collect() is an action that returns the entire data set in an Array to the driver.

PySpark withColumn()

PySpark `withColumn()` is a transformation function of DataFrame which is used to change the value, convert the datatype of an existing column, create a new column, and many more.

```
import pyspark
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lit
from pyspark.sql.types import StructType, StructField,
StringType,IntegerType

spark = SparkSession.builder.appName('SparkByExamples.com').getOrCreate()

data = [('James','','Smith','1991-04-01','M',3000),
  ('Michael','Rose','','2000-05-19','M',4000),
  ('Robert','','Williams','1978-09-05','M',4000),
  ('Maria','Anne','Jones','1967-12-01','F',4000),
  ('Jen','Mary','Brown','1980-02-17','F',-1)
]

columns = ["firstname","middlename","lastname","dob","gender","salary"]
df = spark.createDataFrame(data=data, schema = columns)
df.printSchema()
df.show(truncate=False)

df2 = df.withColumn("salary",col("salary").cast("Integer"))
df2.printSchema()
df2.show(truncate=False)

df3 = df.withColumn("salary",col("salary")*100)
df3.printSchema()
df3.show(truncate=False)

df4 = df.withColumn("CopiedColumn",col("salary")* -1)
df4.printSchema()

df5 = df.withColumn("Country", lit("USA"))
df5.printSchema()

df6 = df.withColumn("Country", lit("USA")) \
   .withColumn("anotherColumn",lit("anotherValue"))
df6.printSchema()

df.withColumnRenamed("gender","sex") \
  .show(truncate=False)

df4.drop("CopiedColumn") \
.show(truncate=False)
```

PySpark Distinct to Drop Duplicate Rows

```
# Applying distinct() to remove duplicate rows
distinctDF = df.distinct()

# Applying dropDuplicates() to remove duplicates
df2 = df.dropDuplicates()
# Remove duplicates on selected columns using dropDuplicates()
dropDisDF = df.dropDuplicates(["department","salary"])
```

PySpark Join Types

1. PySpark Join Syntax

PySpark SQL join has a below syntax and it can be accessed directly from DataFrame.

```
# Syntax
join(self, other, on=None, how=None)
```

```
import pyspark
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder.appName('SparkByExamples.com').getOrCreate()

emp = [(1,"Smith",-1,"2018","10","M",3000), \
     (2,"Rose",1,"2010","20","M",4000), \
     (3,"Williams",1,"2010","10","M",1000), \
     (4,"Jones",2,"2005","10","F",2000), \
     (5,"Brown",2,"2010","40","",-1), \
        (6,"Brown",2,"2010","50","",-1) \
   ]
empColumns = ["emp_id","name","superior_emp_id","year_joined", \
         "emp_dept_id","gender","salary"]

empDF = spark.createDataFrame(data=emp, schema = empColumns)
empDF.printSchema()
empDF.show(truncate=False)


dept = [("Finance",10), \
     ("Marketing",20), \
     ("Sales",30), \
     ("IT",40) \
   ]
deptColumns = ["dept_name","dept_id"]
deptDF = spark.createDataFrame(data=dept, schema = deptColumns)
```

```
deptDF.printSchema()
deptDF.show(truncate=False)

empDF.join(deptDF,empDF.emp_dept_id ==  deptDF.dept_id,"inner") \
      .show(truncate=False)

empDF.join(deptDF,empDF.emp_dept_id ==  deptDF.dept_id,"outer") \
     .show(truncate=False)
empDF.join(deptDF,empDF.emp_dept_id ==  deptDF.dept_id,"full") \
     .show(truncate=False)
empDF.join(deptDF,empDF.emp_dept_id ==  deptDF.dept_id,"fullouter") \
     .show(truncate=False)

empDF.join(deptDF,empDF.emp_dept_id ==  deptDF.dept_id,"left") \
     .show(truncate=False)
empDF.join(deptDF,empDF.emp_dept_id ==  deptDF.dept_id,"leftouter") \
    .show(truncate=False)

empDF.join(deptDF,empDF.emp_dept_id ==  deptDF.dept_id,"right") \
    .show(truncate=False)
empDF.join(deptDF,empDF.emp_dept_id ==  deptDF.dept_id,"rightouter") \
    .show(truncate=False)

empDF.join(deptDF,empDF.emp_dept_id ==  deptDF.dept_id,"leftsemi") \
    .show(truncate=False)

empDF.join(deptDF,empDF.emp_dept_id ==  deptDF.dept_id,"leftanti") \
    .show(truncate=False)

empDF.alias("emp1").join(empDF.alias("emp2"), \
     col("emp1.superior_emp_id") == col("emp2.emp_id"),"inner") \
     .select(col("emp1.emp_id"),col("emp1.name"), \
       col("emp2.emp_id").alias("superior_emp_id"), \
       col("emp2.name").alias("superior_emp_name")) \
    .show(truncate=False)

empDF.createOrReplaceTempView("EMP")
deptDF.createOrReplaceTempView("DEPT")

joinDF = spark.sql("select * from EMP e, DEPT d where e.emp_dept_id == d.dept_id") \
   .show(truncate=False)

joinDF2 = spark.sql("select * from EMP e INNER JOIN DEPT d ON e.emp_dept_id ==
d.dept_id") \
   .show(truncate=False)
```

PySpark Union and UnionAll

```
# Imports
import pyspark
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('SparkByExamples.com').getOrCreate()

simpleData = [("James","Sales","NY",90000,34,10000), \
     ("Michael","Sales","NY",86000,56,20000), \
     ("Robert","Sales","CA",81000,30,23000), \
     ("Maria","Finance","CA",90000,24,23000) \
   ]

columns= ["employee_name","department","state","salary","age","bonus"]
df = spark.createDataFrame(data = simpleData, schema = columns)
df.printSchema()
df.show(truncate=False)

simpleData2 = [("James","Sales","NY",90000,34,10000), \
     ("Maria","Finance","CA",90000,24,23000), \
     ("Jen","Finance","NY",79000,53,15000), \
     ("Jeff","Marketing","CA",80000,25,18000), \
     ("Kumar","Marketing","NY",91000,50,21000) \
   ]
columns2= ["employee_name","department","state","salary","age","bonus"]

df2 = spark.createDataFrame(data = simpleData2, schema = columns2)

df2.printSchema()
df2.show(truncate=False)

unionDF = df.union(df2)
unionDF.show(truncate=False)
disDF = df.union(df2).distinct()
disDF.show(truncate=False)

unionAllDF = df.unionAll(df2)
unionAllDF.show(truncate=False)
```

PySpark Random Sample

```
df=spark.range(100)
print(df.sample(0.06).collect())
//Output: [Row(id=0), Row(id=2), Row(id=17), Row(id=25), Row(id=26),
Row(id=44), Row(id=80)]
```

```
print(df.sample(0.1,123).collect())
//Output: 36,37,41,43,56,66,69,75,83

print(df.sample(0.1,123).collect())
//Output: 36,37,41,43,56,66,69,75,83

print(df.sample(0.1,456).collect())
//Output: 19,21,42,48,49,50,75,80
```

PySpark fillna() & fill()

```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .master("local[1]") \
    .appName("SparkByExamples.com") \
    .getOrCreate()

filePath="resources/small_zipcode.csv"
df = spark.read.options(header='true', inferSchema='true') \
          .csv(filePath)

df.printSchema()
df.show(truncate=False)


df.fillna(value=0).show()
df.fillna(value=0,subset=["population"]).show()
df.na.fill(value=0).show()
df.na.fill(value=0,subset=["population"]).show()


df.fillna(value="").show()
df.na.fill(value="").show()

df.fillna("unknown",["city"]) \
    .fillna("",["type"]).show()

df.fillna({"city": "unknown", "type": ""}) \
    .show()

df.na.fill("unknown",["city"]) \
    .na.fill("",["type"]).show()

df.na.fill({"city": "unknown", "type": ""}) \
    .show()
```

PySpark partitionBy()

PySpark partition is a way to split a large dataset into smaller datasets based on one or more partition keys.

```
# partitionBy() Example
df.write.option("header",True)  \
         .partitionBy("state")  \
         .mode("overwrite")  \
         .csv("/tmp/zipcodes-state")

#partitionBy() multiple columns
df.write.option("header",True)  \
         .partitionBy("state","city")  \
         .mode("overwrite")  \
         .csv("/tmp/zipcodes-state")

#Use repartition() and partitionBy() together
dfRepart.repartition(2)
         .write.option("header",True)  \
         .partitionBy("state")  \
         .mode("overwrite")  \
         .csv("c:/tmp/zipcodes-state-more")

# Reading from partitioned data
dfSinglePart=spark.read.option("header",True)  \
              .csv("c:/tmp/zipcodes-state/state=AL/city=SPRINGVILLE")
dfSinglePart.printSchema()
dfSinglePart.show()

# Read from partitioned data using sql
parqDF  =  spark.read.option("header",True)  \
                     .csv("/tmp/zipcodes-state")
parqDF.createOrReplaceTempView("ZIPCODE")
spark.sql("select * from ZIPCODE  where state='AL' and city = 'SPRINGVILLE'")  \
     .show()
```

PySpark Aggregate Functions

```
# Imports
import pyspark
from pyspark.sql import SparkSession
from pyspark.sql.functions import approx_count_distinct,collect_list
from pyspark.sql.functions import
collect_set,sum,avg,max,countDistinct,count
from pyspark.sql.functions import first, last, kurtosis, min, mean,
skewness
from pyspark.sql.functions import stddev, stddev_samp, stddev_pop,
sumDistinct
from pyspark.sql.functions import variance,var_samp,  var_pop

spark = SparkSession.builder.appName('SparkByExamples.com').getOrCreate()

simpleData = [("James", "Sales", 3000),
     ("Michael", "Sales", 4600),
     ("Robert", "Sales", 4100),
     ("Maria", "Finance", 3000),
     ("James", "Sales", 3000),
     ("Scott", "Finance", 3300),
     ("Jen", "Finance", 3900),
     ("Jeff", "Marketing", 3000),
     ("Kumar", "Marketing", 2000),
     ("Saif", "Sales", 4100)
  ]
schema = ["employee_name", "department", "salary"]

df = spark.createDataFrame(data=simpleData, schema = schema)
df.printSchema()
df.show(truncate=False)

print("approx_count_distinct: " + \
       str(df.select(approx_count_distinct("salary")).collect()[0][0]))

print("avg: " + str(df.select(avg("salary")).collect()[0][0]))

df.select(collect_list("salary")).show(truncate=False)

df.select(collect_set("salary")).show(truncate=False)

df2 = df.select(countDistinct("department", "salary"))
df2.show(truncate=False)
print("Distinct Count of Department & Salary: "+str(df2.collect()[0][0]))

print("count: "+str(df.select(count("salary")).collect()[0]))
```

```
df.select(first("salary")).show(truncate=False)
df.select(last("salary")).show(truncate=False)
df.select(kurtosis("salary")).show(truncate=False)
df.select(max("salary")).show(truncate=False)
df.select(min("salary")).show(truncate=False)
df.select(mean("salary")).show(truncate=False)
df.select(skewness("salary")).show(truncate=False)
df.select(stddev("salary"), stddev_samp("salary"), \
     stddev_pop("salary")).show(truncate=False)
df.select(sum("salary")).show(truncate=False)
df.select(sumDistinct("salary")).show(truncate=False)
df.select(variance("salary"),var_samp("salary"),var_pop("salary")) \
  .show(truncate=False)
```

PySpark Window Functions

```
import pyspark
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('SparkByExamples.com').getOrCreate()

simpleData = (("James", "Sales", 3000), \
     ("Michael", "Sales", 4600),   \
     ("Robert", "Sales", 4100),    \
     ("Maria", "Finance", 3000),   \
     ("James", "Sales", 3000),     \
     ("Scott", "Finance", 3300),   \
     ("Jen", "Finance", 3900),     \
     ("Jeff", "Marketing", 3000), \
     ("Kumar", "Marketing", 2000),\
     ("Saif", "Sales", 4100)  \
   )

columns= ["employee_name", "department", "salary"]

df = spark.createDataFrame(data = simpleData, schema = columns)

df.printSchema()
df.show(truncate=False)

from pyspark.sql.window import Window
from pyspark.sql.functions import row_number
windowSpec  = Window.partitionBy("department").orderBy("salary")

df.withColumn("row_number",row_number().over(windowSpec)) \
     .show(truncate=False)

from pyspark.sql.functions import rank
df.withColumn("rank",rank().over(windowSpec)) \
     .show()

from pyspark.sql.functions import dense_rank
df.withColumn("dense_rank",dense_rank().over(windowSpec)) \
     .show()

from pyspark.sql.functions import percent_rank
df.withColumn("percent_rank",percent_rank().over(windowSpec)) \
     .show()

from pyspark.sql.functions import ntile
df.withColumn("ntile",ntile(2).over(windowSpec)) \
     .show()
```

```
from pyspark.sql.functions import cume_dist
df.withColumn("cume_dist",cume_dist().over(windowSpec)) \
   .show()

from pyspark.sql.functions import lag
df.withColumn("lag",lag("salary",2).over(windowSpec)) \
      .show()

from pyspark.sql.functions import lead
df.withColumn("lead",lead("salary",2).over(windowSpec)) \
    .show()

windowSpecAgg  = Window.partitionBy("department")
from pyspark.sql.functions import col,avg,sum,min,max,row_number
df.withColumn("row",row_number().over(windowSpec)) \
  .withColumn("avg", avg(col("salary")).over(windowSpecAgg)) \
  .withColumn("sum", sum(col("salary")).over(windowSpecAgg)) \
  .withColumn("min", min(col("salary")).over(windowSpecAgg)) \
  .withColumn("max", max(col("salary")).over(windowSpecAgg)) \
  .where(col("row")==1).select("department","avg","sum","min","max") \
  .show()
```

PySpark SQL Date and Timestamp Functions

```
#current_date()
df.select(current_date().alias("current_date")
   ).show(1)

#Result
+------------+
|current_date|
+------------+
|   2021-02-22|
+------------+
```

```
#date_format()
df.select(col("input"),
     date_format(col("input"),  "MM-dd-yyyy").alias("date_format")
   ).show()

#Result
+----------+-----------+
|input      |date_format|
+----------+-----------+
|2020-02-01|  02-01-2020|
|2019-03-01|  03-01-2019|
```

```
#to_date()
df.select(col("input"),
     to_date(col("input"),  "yyy-MM-dd").alias("to_date")
   ).show()

#Result
+----------+----------+
|      input|    to_date|
+----------+----------+
|2020-02-01|2020-02-01|
|2019-03-01|2019-03-01|
```

```
#datediff()
df.select(col("input"),
     datediff(current_date(),col("input")).alias("datediff")
   ).show()

#Result
+----------+--------+
|      input|datediff|
+----------+--------+
|2020-02-01|       387|
|2019-03-01|       724|
```

```
#months_between()
df.select(col("input"),
     months_between(current_date(),col("input")).alias("months_between")
   ).show()

#Result
+----------+--------------+
|      input|months_between|
+----------+--------------+
|2020-02-01|     12.67741935|
|2019-03-01|     23.67741935|
```

```
df.select(col("input"),
      year(col("input")).alias("year"),
      month(col("input")).alias("month"),
      next_day(col("input"),"Sunday").alias("next_day"),
      weekofyear(col("input")).alias("weekofyear")
   ).show()

#Result
+----------+----+-----+----------+----------+
|      input|year|month|  next_day|weekofyear|
+----------+----+-----+----------+----------+
|2020-02-01|2020|      2|2020-02-02|         5|
|2019-03-01|2019|      3|2019-03-03|         9|
```

```
data=[["1","02-01-2020 11 01 19 06"],["2","03-01-2019 12 01 19 406"],["3","03-01-2021 12 01 19 406"]]
df2=spark.createDataFrame(data,["id","input"])
df2.show(truncate=False)

#Result
+---+----------------------+
|id |input                 |
+---+----------------------+
|1  |02-01-2020 11 01 19 06 |
```

```python
from pyspark.sql import SparkSession

# Initialize Spark session
spark = SparkSession.builder.appName("example").getOrCreate()

# Example DataFrame
data = [("James", "Sales", 3000),
        ("Michael", "Sales", 4600),
        ("Robert", "Sales", 4100),
        ("Maria", "Finance", 3000),
        ("James", "Sales", 3000),
        ("Scott", "Finance", 3300),
        ("Jen", "Finance", 3900),
        ("Jeff", "Marketing", 3000),
        ("Kumar", "Marketing", 2000)]
columns = ["EmployeeName", "Department", "Salary"]

df = spark.createDataFrame(data, columns)

# Register DataFrame as a temporary view
df.createOrReplaceTempView("employee_view")

# Convert the temporary view back to a DataFrame
df_from_view = spark.sql("SELECT * FROM employee_view")

# Convert the DataFrame to an RDD
rdd_from_df = df_from_view.rdd

# Show the content of the DataFrame
df_from_view.show()

# Show the content of the RDD
print(rdd_from_df.collect())
```

PySpark foreach()

PySpark `foreach()` is an action operation that is available in RDD, DataFram to iterate/loop over each element in the DataFrmae

When `foreach()` applied on PySpark DataFrame, it executes a function specified in for each element of DataFrame. This operation is mainly used if you wanted to manipulate accumulators, save the DataFrame results to RDBMS tables, Kafka topics, and other external sources.

```
from pyspark.sql import SparkSession

# Create SparkSession
spark = SparkSession.builder.appName('SparkByExamples.com') \
                      .getOrCreate()

# Prepare Data
columns = ["Seqno","Name"]
data = [("1", "john jones"),
      ("2", "tracey smith"),
      ("3", "amy sanders")]

# Create DataFrame
df = spark.createDataFrame(data=data,schema=columns)
df.show()

# foreach() Example
def f(df):
      print(df.Seqno)
df.foreach(f)
```

```
# foreach() with accumulator Example
accum=spark.sparkContext.accumulator(0)
df.foreach(lambda x:accum.add(int(x.Seqno)))
print(accum.value) #Accessed by driver
```